

Table of Contents

[UltraVIDCDiffMeterDefTopic](#)

[UltraVIDCDiffMeter_about.1.2](#)

[UltraVIDCDiffMeter_about.1.3](#)

[UltraVIDCDiffMeter_about.1.4](#)

[UltraVIDCDiffMeter_about.1.5](#)

[UltraVIDCDiffMeter_apps.4.01](#)

[UltraVIDCDiffMeter_apps.4.02](#)

[UltraVIDCDiffMeter_apps.4.03](#)

[UltraVIDCDiffMeter_apps.4.04](#)

[UltraVIDCDiffMeter_apps.4.05](#)

[UltraVIDCDiffMeter_apps.4.06](#)

[UltraVIDCDiffMeter_apps.4.07](#)

[UltraVIDCDiffMeter_apps.4.08](#)

[UltraVIDCDiffMeter_apps.4.09](#)

[UltraVIDCDiffMeter_apps.4.10](#)

[UltraVIDCDiffMeter_program.3.01](#)

[UltraVIDCDiffMeter_program.3.02](#)

[UltraVIDCDiffMeter_program.3.03](#)

[UltraVIDCDiffMeter_program.3.04](#)

[UltraVIDCDiffMeter_program.3.05](#)

[UltraVIDCDiffMeter_program.3.06](#)

[UltraVIDCDiffMeter_program.3.07](#)

[UltraVIDCDiffMeter_program.3.08](#)

[UltraVIDCDiffMeter_program.3.09](#)

[UltraVIDCDiffMeter_program.3.10](#)

[UltraVIDCDiffMeter_program.3.11](#)

[UltraVIDCDiffMeter_program.3.12](#)

[UltraVIDCDiffMeter_program.3.13](#)

[UltraVIDCDiffMeter_program.3.14](#)

UltraVIDCDiffMeter_program.3.15
UltraVIDCDiffMeter_program.3.16
UltraVIDCDiffMeter_program.3.17
UltraVIDCDiffMeter_program.3.18
UltraVIDCDiffMeter_program.3.19
UltraVIDCDiffMeter_too.2.01
UltraVIDCDiffMeter_too.2.02
UltraVIDCDiffMeter_too.2.03
UltraVIDCDiffMeter_too.2.04
UltraVIDCDiffMeter_too.2.05
UltraVIDCDiffMeter_too.2.06
UltraVIDCDiffMeter_too.2.07
UltraVIDCDiffMeter_too.2.08
UltraVIDCDiffMeter_too.2.09
UltraVIDCDiffMeter_too.2.10
UltraVIDCDiffMeter_too.2.11
UltraVIDCDiffMeter_too.2.12
UltraVIDCDiffMeter_too.2.13
UltraVIDCDiffMeter_too.2.14
UltraVIDCDiffMeter_too.2.15
UltraVIDCDiffMeter_too.2.16
UltraVIDCDiffMeter_too.2.17
UltraVIDCDiffMeter_too.2.18
UltraVIDCDiffMeter_too.2.19
UltraVIDCDiffMeter_too.2.20

UltraVIDCDiffMeterDefTopic

<

>

About the UltraVI80 DCDiffMeter

About the UltraVI80 DCDiffMeter

The UltraVI80 DC Differential Voltmeter (DCDiffMeter) is a DC instrument that makes small voltage measurements on a large common-mode signal. In addition, each differential voltmeter has three lower-voltage ranges with improved specifications over the channel DCVI meters. The UltraVI80 DCDiffMeter can also be used with the DCTime Measurement Units on the UltraVI80 instrument board, through the asynchronous trigger lines, to make measurements. The UltraVI80 instrument includes eight DC Differential Voltmeters. The following topics are included:

- [Features](#)
- [Safety Information](#)
- [Key Terms](#)
- [UVI80 DCDiffMeter Software Licensing Requirements](#)

DCDiffMeter information includes the following sections:

- [UltraVI80 DCDiffMeter Theory of Operation](#)
- [UltraVI80 DCDiffMeter Programming](#)
- [UltraVI80 DCDiffMeter Examples](#)

Related Topics

- [UltraVI80 Debug Displays](#)
- [UltraVI80 Specifications](#)

- [DCDiffMeter](#) (VBT syntax)

- [UltraVI80 DIB Design Guidelines](#)

For more information on UltraVI80 instruments see:

- [UltraVI80 Instrument Board](#)

- [About the UltraVI80 DCTime](#)

- [About the UltraVI80 DCVI](#)

For information on DC30 and DC75 DCDiffMeters, see [About the DCDiffMeter](#).

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_about.1.2

<	>
-----------------	-----------------

About the UltraVI80 DCDiffMeter : Features

Features

The following are the UltraVI80 DC Differential Voltmeter features:

- Eight DC Differential Voltmeters per UltraVI80 instrument
 - Accessed through VI channel sense lines
 - Can connect to one of two DGS lines available per group of 20 VI channels
 - Precision voltage measurement capability
- 2 DC Differential Voltmeters per 20 channel slice
- Differential voltage: 7 V max.
- Common mode input voltage: -2V to 7V
- Capture memory depth: 4M samples

To improve accuracy, two separate analog-to-digital converters are included, available in two modes of operation:

- High Accuracy

- 24-bit oversampled SAR ADC
- 1K strobes memory
- Waveform captures not supported
- Sample rate 500sps to 100ksps, adjustable
- High Speed
- 16-bit ADC
- 1k strobes memory
- 4M waveform samples memory
- Sample rate 508.63sps to 8.33Msps, adjustable with 120ns resolution
- All DCDiffMeters have:
- Measure voltage ranges of 1.4V, 3.5V, and 7V
- Hardware averaging: 2, 4, 8, 16, 32, 64, 128, and 256 samples can be averaged

The following table compares DC Differential Voltmeter features for DC30, DC75, and UltraVI80.

Comparison of DC Differential Voltmeter Features			
Feature	DC30	DC75	UltraVI80
Number of instruments	2 per instrument assembly	2 per instrument assembly	2 per slice = 4 per half board = 8 per instrument assembly
Operation modes	Fixed	Fixed	High Speed, High Accuracy

Voltage ranges	0.05V, 0.1V, 0.25V, 0.5V, 1V, 2.5V, 5V	0.05V, 0.1V, 0.25V, 0.5V, 2.5V, 5V	1.4V, 3.5V, 7V
Sample rate	781.25sps to 100ksps	781.25sps to 100ksps	508.63sps to 8.33Msps high speed 500sps to 100ksps high accuracy
Sample size	512	512	4M high speed 1K high accuracy
Instrument-Initiated Move, Background DSP	No	No	Yes
DMA capable	Yes	Yes	Yes
Hardware averaging	No	No	Yes

For more information, see [UltraVI80 Specifications](#).

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_about.1.3



About the UltraVI80 DCDiffMeter : Safety Information

Safety Information

Follow general safety procedures when operating UltraVI80 instrument boards. Read all system and instrument safety instructions before operating the test system. Only trained personnel should operate the test system. For more information, see [Test System Safety](#) in the [Safety](#) section.

Warning:

Even if a DCVI instrument is in a programmed state that is not forcing current or voltage, this does not imply any sense of safety. If the instrument is powered on, the potential of danger exists at the DIB.

Tester Cover

Keep the test head cover in place when the tester is in operation. If the test head cover is removed, an interlock is activated powering down the test system. Only trained and authorized personnel should operate the tester with the test head cover removed.

Warning:

Even with Gate Off, this does not imply any sense of safety. Voltage can exist at the channel output. Follow proper safety rules before handling a DIB. For proper handling of a DIB, see [About Operator and Safety Information](#) in the [Safety](#) section.

DIB Handling

Circuitry on a DIB can store hazardous potential depending on the components on the DIB (capacitors). Use proper care when handling a DIB even after the test system is powered down. Since DIBs are designed by customers, the test system cannot track if a DIB has safely dissipated stored energies.

Warning:

Be sure that any power storage devices on a DIB have discharged stored potential before handling the DIB or personnel injury can occur.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

UltraVIDCDiffMeter_about.1.4

<	>
-----------------	-----------------

About the UltraVI80 DCDiffMeter : Key Terms

Key Terms

capture	The act of a meter making measurements on a signal and saving the samples to memory.
Capture memory	The number of samples measured in a capture that can be stored. DCDiffMeters can capture up to 4M samples.
DGS	Device ground sense is a sense signal that tracks the voltage of ground on a DIB.
force line	Wiring that carries programmed output from a DCVI to a DUT pin.
sense line	High impedance wiring used to measure the output of the force line. Also used to make connections from the DCDiffMeter and DCTime to a DUT pin. This line is part of the control loop.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_about.1.5

<	>
-----------------	-----------------

About the [UltraVI80 DCDiffMeter](#) : UVI80 DCDiffMeter Software Licensing Requirements

UVI80 DCDiffMeter Software Licensing Requirements

IG-XL enforces counted licenses for each UltraVI80 DCVI channel, and each DCVI channel is counted as one license. However, DCDiffMeters do not count for licensing.

For more information, see:

- | | |
|--|---|
| | <ul style="list-style-type: none">UltraVI80 Software Licensing Requirements |
| | <ul style="list-style-type: none">IG-XL Licensing |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_apps.4.01

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Examples

UltraVI80 DCDiffMeter Examples

This section provides device-based test program examples for the UltraVI80 DC Differential Voltmeter (DCDiffMeter).

The following topics are available:

- UltraVI80 DCDiffMeter Delta Voltage Measurement Test
- UltraVI80 DCDiffMeter VBT Examples

For more information, see:

- DCDiffMeter (VBT Syntax Reference)
- UltraVI80 DCDiffMeter Programming

<	>
-----------------	-----------------

UltraVIDCDiffMeter_apps.4.02

<	>
-----------------	-----------------

[UltraVI80 DCDiffMeter Examples](#) : UltraVI80 DCDiffMeter VBT Examples

UltraVI80 DCDiffMeter VBT Examples
The following VBT examples are included:

- | | |
|---|---|
| • | Specify Low Side of the DCDiffMeter |
| • | Set Up the DCDiffMeter |
| • | Program Individual Parameters to Program a DCDiffMeter PSet |
| • | Program Set Statements to Program a DCDiffMeter PSet |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_apps.4.03

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Examples : UltraVI80 DCDiffMeter VBT Examples : Specify Low Side of the DCDiffMeter

```
Specify Low Side of the DCDiffMeter
Public Function DiffMeterLowPin()
' Specify the low side of the DCDiffMeter
TheHdw.DCDiffMeter.Pins("ch1hi").LowSide.Pins = ("ch1lo")
End Function
```

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_apps.4.04

<	>
-----------------	-----------------

[UltraVI80 DCDiffMeter Examples](#) : [UltraVI80 DCDiffMeter VBT Examples](#) : Set Up the DCDiffMeter

Set Up the DCDiffMeter

Public Function DiffMeterLowPin()

' Set up the DCDiffMeter

With TheHdw.DCDiffMeter.Pins("ch1hi")

 .Connect tIDCDiffMeterConnectDefault

 .VoltageRange = 3.5

End With

End Function

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_apps.4.05

	
---	---

[UltraVI80 DCDiffMeter Examples](#) : [UltraVI80 DCDiffMeter VBT Examples](#) : Program Individual Parameters to Program a DCDiffMeter PSet

[Program Individual Parameters to Program a DCDiffMeter PSet](#)

[Public Function DCDiffMeterPSet\(\)](#)

[' Specify the low side of the DCDiffMeter](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").LowSide.Pins = ("DCDiffLow")

[' Connect the DCDiffMeter to the DUT](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").Connect

[' Add a PSet to the PSet collection](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Add ("MyPSet")

[' Get PSet from the PSet collection and program the PSet properties](#)

With TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Item("MyPSet")

.Capture.ExtraSamples = 3

.Capture.SampleRate = 10000

.Capture.SampleSize = 100

.VoltageRange = 7

.HardwareAverage = 64

End With

[' Apply PSet to hardware](#)

TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Item("MyPSet").Apply

[End Function](#)

	
---	---

UltraVIDCDiffMeter_apps.4.06

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Examples : UltraVI80 DCDiffMeter VBT Examples : Program Set Statements to Program a DCDiffMeter PSet

Program Set Statements to Program a DCDiffMeter PSet

```
Public Function DCDiffMeterPSetSetStatement()  
' Specify the low side of the DCDiffMeter  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").LowSide.Pins = ("DCDiffLow")  
' Connect the DCDiffMeter to the DUT  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").Connect  
' Add a PSet to the PSet collection and program with a Set statement  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Add("MyPSet").Set 7, False, 10000, 100, 3, 64  
' Apply PSet to hardware  
TheHdw.DCDiffMeter.Pins("DCDiffHigh").PSets.Item("MyPSet").Apply  
End Function
```

<	>
-----------------	-----------------

UltraVIDCDiffMeter_apps.4.07

<	>
-----------------	-----------------

[UltraVI80 DCDiffMeter Examples](#) : UltraVI80 DCDiffMeter Delta Voltage Measurement Test

UltraVI80 DCDiffMeter Delta Voltage Measurement Test

This test makes a differential measurement on the voltage regulator.

The required programming steps are:

- | | |
|---|-------------------------------------|
| • | Set Up the DUT |
| • | Set Up DataTool |
| • | Create the VBT Code |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_apps.4.08

<

>

UltraVI80 DCDiffMeter Examples : UltraVI80 DCDiffMeter Delta Voltage Measurement Test : Set Up the DUT

Set Up the DUT

The figure shows the DUT for this test.



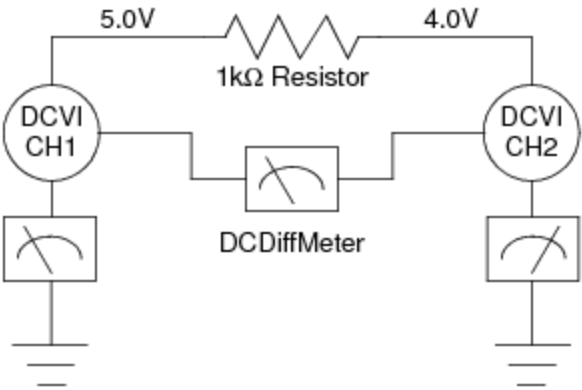
DUT Block Diagram

The table lists the specifications for this test.

DUT Specification Sheet

Pin Name	Pin Function
ch1	Connects to channel 1
ch2	Connects to channel 2

This figure shows DUT connections and DIB wiring required for this test.



DIB Layout

<		>	
	2015 Teradyne, Inc. All rights reserved.		

UltraVIDCDiffMeter_apps.4.09

<

>

UltraVI80 DCDiffMeter Examples : UltraVI80 DCDiffMeter Delta Voltage Measurement Test : Set Up DataTool

Set Up DataTool

This test uses the following DataTool sheets and data:

- Pin Map Sheet
- Channel Map Sheet
- Test Instances Sheet
- Flow Table Sheet

Pin Map Sheet

Pin Map			
Group Name	Pin Name	Type	Comment
	ch1	Analog	
	ch2	Analog	
	ch1hi	Analog	
	ch1lo	Analog	

Channel Map Sheet

Channel Map				
DIB ID:		View Mode: Signal		
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
ch1		DCVI	23.sense1	
ch2		DCVI	23.sense2	
ch1hi		DCDiffMeter	23.sense1	
ch1lo		DCDiffMeter	23.sense2	

Test Instances Sheet

Test Instances			
Test Name		Instance	
Type	Name	Called As	
DCDiffMeterMeasurement	VBT	DiffMeterMeasure	

Flow Table Sheet

Flow Table						
Gate			Command			
Label	Enable	Job	Part	Env	Opcode	Parameter
					Test	DCDiffMeterMeasurement

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_apps.4.10

	
---	---

[UltraVI80 DCDiffMeter Examples : UltraVI80 DCDiffMeter Delta Voltage Measurement Test](#) : Create the VBT Code

Create the VBT Code

The following VBT code performs this test. Cut and paste it into the VBT editor.

[Public Function](#) DiffMeterMeasure()

' Program the DCVI connected to the Input pin

With TheHdw.DCVI.Pins("ch1")

.Connect

.Gate = False

.Mode = tlDCVIModeVoltage

.Voltage = 5

.VoltageRange.Autorange = True

.Current = 0.1

.CurrentRange.Autorange = True

End With

' Program the DCVI connected to the Output pin

With TheHdw.DCVI.Pins("ch2")

.Connect

.Gate = False

.Mode = tlDCVIModeVoltage

.Voltage = 5

.VoltageRange.Autorange = True

.Current = 0.1

.CurrentRange.Autorange = True

End With

' Gate on all DCVIs

```
TheHdw.DCVI.Pins("ch1,ch2").Connect
```

' Specify the low side of the DCDiffMeter

```
TheHdw.DCDiffMeter.Pins("ch1hi").LowSide.Pins = ("ch1lo")
```

' Set up the DCDiffMeter

```
With TheHdw.DCDiffMeter.Pins("ch1hi")  
    .Connect tlDCDiffMeterConnectDefault  
    .VoltageRange = 3.5
```

```
End With
```

' Program a wait time

```
TheHdw.Wait (0.001)
```

' Program the DCDiffmeter to make a measurement

```
Dim Results As New PinListData
```

```
Results = TheHdw.DCDiffMeter.Pins("ch1hi").Read(tlStrobe, 1, 100)
```

```
Call TheExec.Flow.TestLimit(resultVal:=Results, ForceResults:=tlForceFlow)
```

' Gate off and disconnect the DCVI

```
With TheHdw.DCVI.Pins("ch1,ch2")  
    .Mode = tlDCVIModeVoltage  
    .Gate = False  
    .Disconnect
```

```
End With
```

```
End Function
```

<	>	
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_program.3.01

<	>
--------------------------------------	--------------------------------------

UltraVI80 DCDiffMeter Programming

UltraVI80 DCDiffMeter Programming

This section describes programming of an UltraVI80 DC Differential Voltmeter (DCDiffMeter). The following topics are available:

- [DCDiffMeter DataTool Programming](#)
- [Connecting a DCDiffMeter](#)
- [DCDiffMeter Modes](#)
- [Onboard Asynchronous Triggering of a DCDiffMeter](#)
- [Programming a DCDiffMeter Waveform Capture](#)
- [Using DCDiffMeter in the Mixed Signal Workshop](#)
- [Visual Basic for Test](#)
- [UltraVI80 DCDiffMeter Reset Values](#)
- [UltraVI80 DCDiffMeter Pattern Control](#)

For more information, see:

- [DCDiffMeter](#) (VBT Syntax Reference)
- [UltraVI80 DCDiffMeter Examples](#)

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_program.3.02

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : DCDiffMeter DataTool Programming

DCDiffMeter DataTool Programming

This topic provides instructions on setting up parameters in worksheets specific to the DCDiffMeter. The following topics are included:

- Programming the Pin Map
- Programming the Channel Map

For more information on DataTool setup and programming, see [About DataTool](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_program.3.03

<	>
-----------------	-----------------

[UltraVI80 DCDiffMeter Programming](#) : [DCDiffMeter DataTool Programming](#) : Programming the Pin Map

Programming the Pin Map

A pin map defines the pins of a device and the type of each pin. You can enter any pin type for a DCDiffMeter. The recommended selection is:

Analog	This type is used for the DCDiffMeter.
--------	--

For more information, see [Pin Map Sheet](#) in the DataTool section.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_program.3.04

<

>

UltraVI80 DCDiffMeter Programming : DCDiffMeter DataTool Programming : Programming the Channel Map

Programming the Channel Map

An UltraVI80 instrument board can support 250 sites. For each pin that is connected to a DCDiffMeter, an entry is required in the PinName, Type and each Site column. Type selection for DCDiffMeter instrument channels is:

DCDiffMeter	Sense line DIB address connection to a DCDiffMeter. In a channel map, both the High Side and the Low Side must have DCDiffMeter defined as the channel type.
-------------	--

For more information, see Channel Map Sheet in the DataTool section.

☒ **Note:**

You must specify both the high and low side in the channel map.
In the Site columns, the channel map is programmed with the DIB connector number and the sense line. The DIB connector location is dependent on tester configuration. The following table shows examples of valid entries:

UltraVI80 Channel Map Entries			
Pin Name	Type	Site	Comment
HighSide	DCDiffMeter	23.sense1	Channel 1 at DIB slot 23.
LowSide	DCDiffMeter	23.sense2 or 23.dgs2	Channel 2 at DIB slot 23 or DGS 2 at DIB slot 23.

☒ **Note:**

5/16/26, 5:50 PM

Combined Documentation

UltraVI80 DGS assignment rules differ from the DC30/DC75. Only 2 DGS are available per slice of 20 channels. For more information, see [UltraVI80 DIB Design Guidelines](#).
You can see this information in the file <installdir>\tester\CurrentChannelMap.txt.

< >		
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_program.3.05

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : Connecting a DCDiffMeter

Connecting a DCDiffMeter

A DCDiffMeter has both a high and low side that are required to be connected. Connections to the DUT are made through the channel sense lines, or the low side can be connected to a DGS. The connections are described in the following topics:

- Connecting a DCDiffMeter High and Low Side to a Sense Line
- Connecting a DCDiffMeter Low Side to DGS

For more information, see:

- DCDiffMeter Subbus Connections
- DCDiffMeter DGS

Note:

If the Force and Sense lines of a DCVI channel are shorted at the DIB and a DCDiffMeter is connected to the Sense line, measurements will be affected by the bleeder resistor. For more information about removing this effect, see DCVI Bleeder Resistor.

<	>
-----------------	-----------------

--	--	--

UltraVIDCDiffMeter_program.3.06

<

>

UltraVI80 DCDiffMeter Programming : [Connecting a DCDiffMeter](#) : Connecting a DCDiffMeter High and Low Side to a Sense Line

[Connecting a DCDiffMeter High and Low Side to a Sense Line](#)
The channel map requires entries similar to the following:

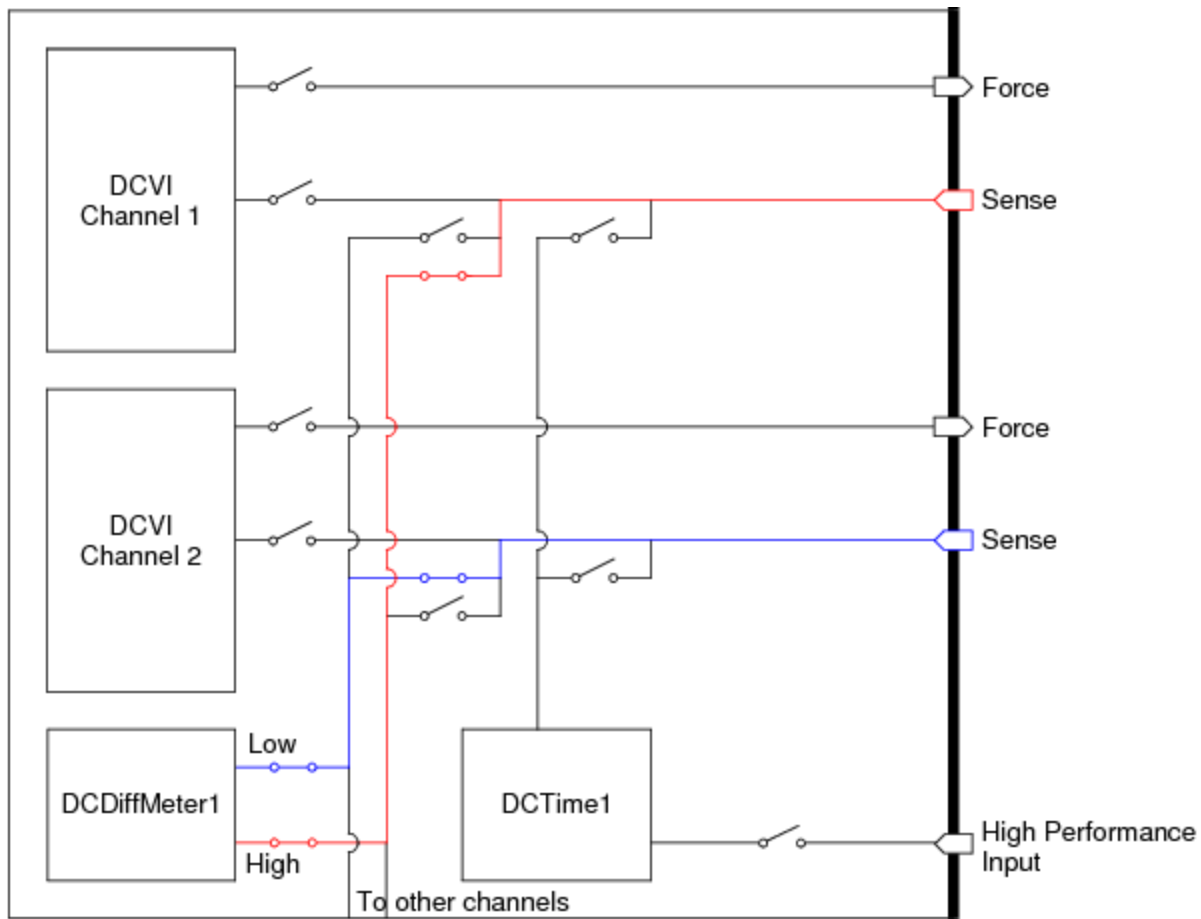
DCDiffMeter Example Channel Map

Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
PinHigh		DCDiffMeter	1.sense1	Channel 1
PinLow		DCDiffMeter	1.sense2	Channel 2

Use the following VBT language to make connections to the DUT:

```
' Declare the lowside pin using the following statement
TheHdw.DCDiffMeter.Pins("PinHigh").LowSide.Pins = "PinLow"
' Connect both pins to the DUT using the following statement
TheHdw.DCDiffMeter.Pins("PinHigh").Connect tIDCDiffMeterConnectDefault
```

The figure shows the connections made:



UltraVI80 DCDiffMeter Connections to Channel Sense Lines

<	>
2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_program.3.07

<

>

UltraVI80 DCDiffMeter Programming : [Connecting a DCDiffMeter](#) : Connecting a DCDiffMeter Low Side to DGS

Connecting a DCDiffMeter Low Side to DGS

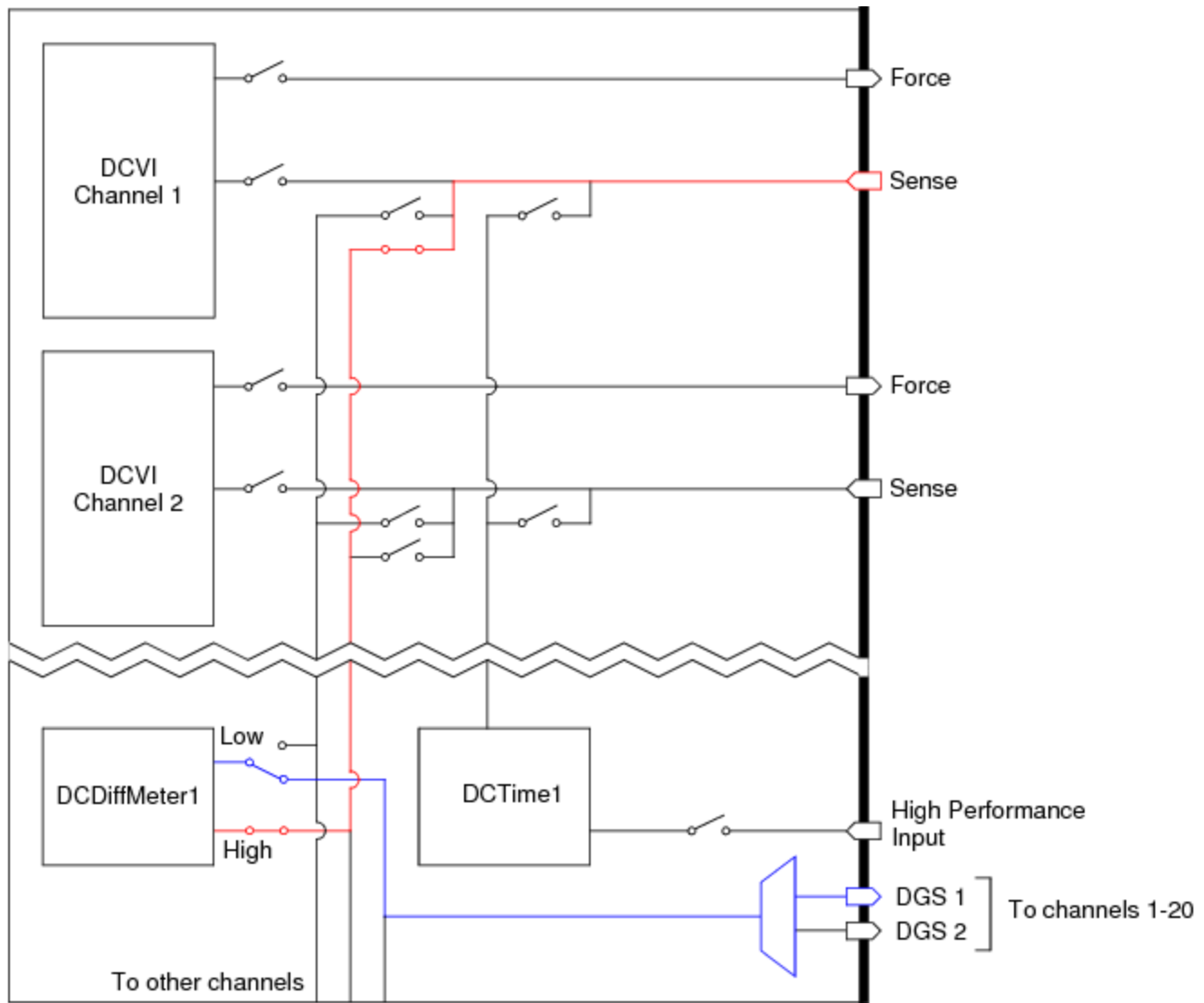
In some tests, you may want to connect the low side of the DCDiffMeter to the channel DGS. To do this the channel map requires entries similar to the following:

DCDiffMeter Example Channel Map				
Device Under Test		Tester Channel		
Pin Name	Package Pin	Type	Site 0	Comment
PinHigh		DCDiffMeter	1.sense1	
PinLowDGS		DCDiffMeter	1.dgs1	

Use the following VBT language to make connections to the DUT:

```
' Declare the Lowside pin using the following statement
TheHdw.DCDiffMeter.Pins("PinHigh").LowSide.Pins = "PinLowDGS"
' Connect both pins using the following statement. PinHigh is connected to the
' DUT through the sense line, and PinLowDGS is connected the channel DGS.
TheHdw.DCDiffMeter.Pins("PinHigh").Connect tIDCDiffMeterConnectDefault
```

The figure shows the connections made:



UltraVI80 DCDiffMeter Low Side Connection to DGS

<	>
2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_program.3.08

<	>
---	---

UltraVI80 DCDiffMeter Programming : DCDiffMeter Modes

DCDiffMeter Modes

The UltraVI80 Differential Voltmeter supports two separate modes of operation: High Accuracy and High Speed. Use the following syntax to choose a mode:

TheHdw.DCDiffMeter.Pins(PinList).MeterMode = tIDCDiffMeterModeHighAccuracy |
tIDCDiffMeterModeHighSpeed

The default value is High Speed.

When the Meter Mode is in High Accuracy mode, IG-XL will only allows you to Strobe the meter. The minimum allowable meter restrobe time is 10s. If you try to issue a Trigger in High Accuracy mode, the following run-time error is returned:

DDM0056: A Capture Trigger can not be executed while in High Accuracy meter mode. To execute a Capture Trigger please change the meter mode to High Speed.

A run-time error is also returned if you try to execute this DCDiffMeter command on a DC30 or DC75 pin.

For more information, see:

- | | |
|---|--|
| • | TheHdw.DCDiffMeter.MeterMode |
|---|--|
- | | |
|---|-----------------------------------|
| • | DCDiffMeter Modes |
|---|-----------------------------------|

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_program.3.09

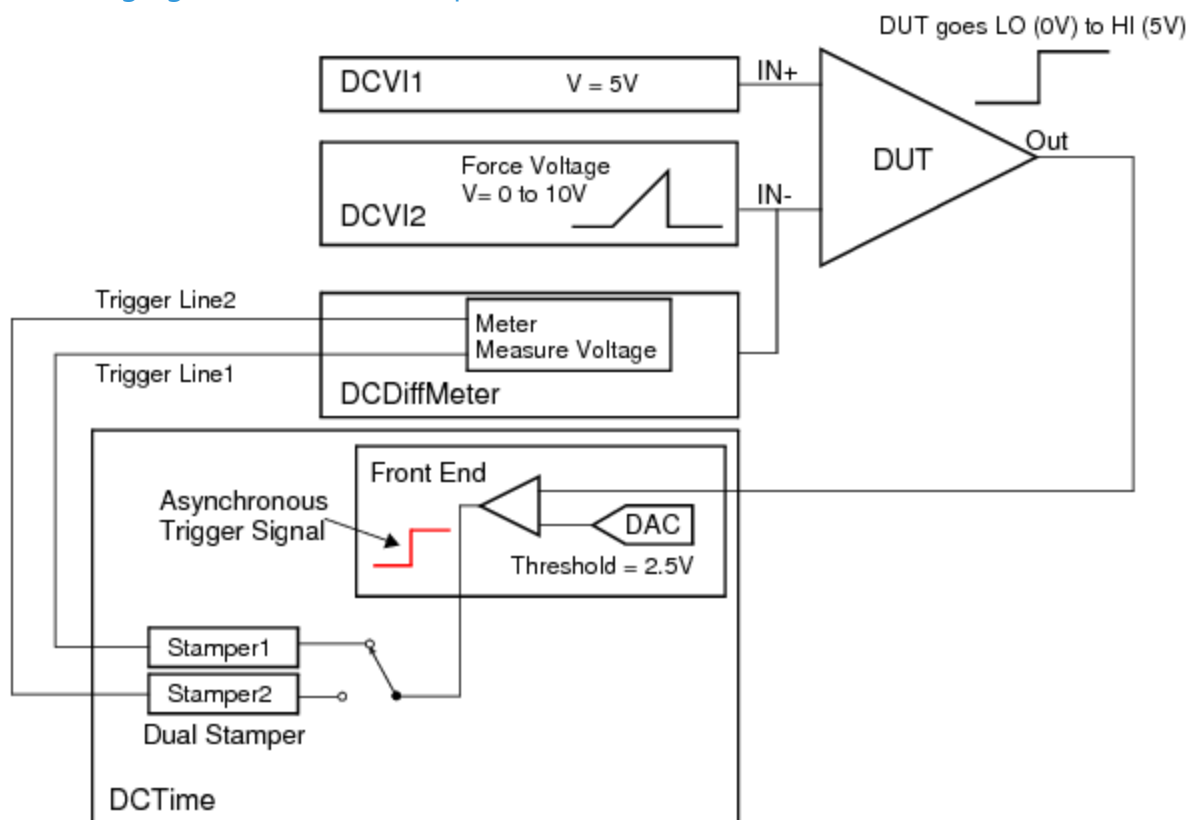


UltraVI80 DCDiffMeter Programming : Onboard Asynchronous Triggering of a DCDiffMeter

Onboard Asynchronous Triggering of a DCDiffMeter

An asynchronous trigger allows you to trigger a DCDiffMeter (or a DCVI meter) on the occurrence of an event using a DCTime instrument. The UltraVI80 has eight asynchronous trigger lines, two per DCTime, that a DCDiffMeter can listen to.

Asynchronous triggering allows you to measure the state change of a device pin. A common example is a threshold test. One device pin receives a voltage ramp from a DCVI. When a certain threshold is crossed, the state of another pin changes. This pin is connected to a DCTime. The following figure shows an example.



Comparator Threshold Test Using a DCDiffMeter

The voltage range, voltage level, and input impedance of the DCTime are programmed per device specifications, and the mode is set to onboard trigger. The DCDiffMeter listens to the trigger line that the DCTime stamper drives when the event occurs.

When the threshold is crossed on the pin, a signal is output from the DCTime which in turn triggers the meter. The measurement made determines the voltage level the ramp was at when the change state occurred.

Making an asynchronous trigger measurement requires both the DCTime front end comparator and a stamper. The front end comparator outputs a signal to the stamper that places a signal onto a trigger line connected to the specified DCDiffMeter.

An UltraVI80 DCDiffMeter can be triggered by a DCTime only if they both reside in the same group of 40 channels (1-40 or 41-80).

For more information, see:

- [Programming Asynchronous Trigger Measurements](#)

- [Asynchronous Trigger](#)

- [Pins.AsynchronousTrigger](#)

- [Onboard Asynchronous Triggering of a DCVI Meter](#)

Select the Trigger Pin

You must specify the pin that triggers a DCDiffMeter to make a measurement. The following VBT syntax selects a DCTime as the asynchronous trigger.

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.SetInput "TrigPin" _  
    , "DCTime", 1
```

[Here](#), TrigPin is the name of the pin that is used to asynchronously trigger the meter. DCTime is the channel type for the pin TrigPin, and 1 indicates which trigger line is used from the specified DCTime.

Set the Trigger Pin Slope

You must program the slope of the trigger signal to the edge that a DCDiffMeter will trigger on. The following syntax programs the slope for the meter.

```
TheHdw.DCDiffMeter.Pins("RampSourcePin").AsynchronousTrigger.Slope _  
    = tISlopePositive
```

Select the Trigger Method

An asynchronous trigger can make a single measurement on the first state change only or can be set to automatically re-enable capturing data each time an event occurs.

To program a DCDiffMeter to be triggered only once (single trigger), use this syntax:

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.Continuous = False
```

To program a DCDiffMeter to be triggered each time a trigger event occurs (continuous trigger), use this syntax:

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.Continuous = True
```

Programming a Trigger Pin

The DCTime listens for an event. On receiving that event, it sends a signal through the trigger line to the instrument programmed to listen to it. The following are required to set up the DCTime:

- **OnBoard Trigger:** This mode notifies the DCTime to send a signal to another instrument using a trigger line. The DCTime listens on its inputs for an event to cross its threshold. Once this occurs, it sends a trigger signal to the instrument programmed to listen to the trigger line. Set this using the following syntax:

```
TheHdw.DCTime.Pins("MyPin").Mode = tlDCTimeModeOnBoardTrigger
```

- **Enabling the Trigger Pin:** The DCDiffMeter must be enabled to make an asynchronous measurement. Enable the trigger pin using the following syntax.

```
TheHdw.DCDiffMeter.Pins("MyPin").AsynchronousTrigger.Enable = True
```

You need not set the trigger pin slope for DCTime.

Other programming required for the DCTime depends on the test setup. For a programming example, see DCVI Meter Asynchronous Trigger Test Example.

The UltraVI80 only supports OnBoard Trigger. It does not support DIB Trigger.

Interaction with the Debug Display

The DCDiffMeter debug display has controls for an asynchronous trigger. These controls allow you to set up parameters for an asynchronous trigger, directly aligning with the VBT language noted above for the same purpose.

Normally, the debug display controls reflect the state of the instrument as set up by this syntax.

However, the Async Enable control is an exception. This control may be cleared if the Asynchronous Trigger is set for a single trigger event (that is, if the Async Continuous control is cleared) and the following test program sequence has occurred:

1. The asynchronous trigger is enabled.
2. A capture has been triggered or the DCDiffMeter has been strobed.

3.

The specified asynchronous trigger event occurs on the DCTime.

When the single trigger event occurs, therefore, the asynchronous trigger is no longer enabled. At this point:

- A call in VBT to .AsynchronousTrigger.Enable will return False.
- The Async Enable control on the DCDiffMeter debug display will be cleared.

For more information, see [DCDiffMeter Debug Display](#).

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_program.3.10



UltraVI80 DCDiffMeter Programming : Programming a DCDiffMeter Waveform Capture

Programming a DCDiffMeter Waveform Capture

Each DCDiffMeter can capture up to 4M samples of a waveform in High Speed mode, with a capture rates of 508.66Hz to 8.34MHz. (Capture is not supported in High Accuracy mode.) The waveform is converted to a series of points along the waveform and stored in capture memory. The points captured are based on the capture rate and the number of samples.

For more information, see [Pins.Capture](#) in the [VBT Syntax Reference](#).

Managing Captures and Trigger Samples

UltraVI80 DCDiffMeter capture samples are stored in external shared memory. However, the use of an external shared memory requires address indexing to retrieve samples on a per channel basis. This differs from direct memory addressing of internal memory used for the DC30/75. Therefore, the UltraVI80 supports up to eight unique capture signal names, each of which is associated with a unique address index (also known as a "capture label"), per DCDiffMeter. The DC30/75 does not have this restriction for capture signal names.

Since the UltraVI80 DCDiffMeter memory can store only eight capture waveform names, IG-XL will wrap around and reuse the same capture label for multiple capture signals. To maintain compatibility with the DC30/75, IG-XL does not return a run-time error if you trigger more than eight unique capture signals per DCDiffMeter. Therefore, you must use some care when managing captures on the UltraVI80.

For example, consider the following example code that adds 10 waveforms. Once the waveforms are added and loaded, each of the 10 waveforms is triggered. This means that capture labels will be wrapped around starting with Cap9.

```
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap1")
```

```
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap2")
```

```
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap3")
```

```
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap4")
```

```
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap5")
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap6")
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap7")
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap8")
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap9")
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Item("Cap10")
```

' Assume all Capture settings have been loaded properly

' First capture to be triggered on this channel

```
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap1") 'Label 0
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap2") 'Label 1
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap3") 'Label 2
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap4") 'Label 3
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap5") 'Label 4
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap6") 'Label 5
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap7") 'Label 6
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap8") 'Label 7
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap9") 'Label 0<-Wrapped!
TheHdw.DCDiffMeter.Pins("DIFF01").Capture.Waveforms.Trigger("Cap10") 'Label 1
```

Due to this behavior, read back capture samples before triggering another capture on the same channel. If you fail to do this, you may encounter the following issues.

- Using a DSP computer (and thus IIM) on a tester with UltraVI80:

The DSP computer uses the slot, channel and site to identify samples. Using the DCDiffMeter code above means that Cap1 and Cap9 would point to same data, as would and Cap2 and Cap10. If you issue multiple triggers on the same channel and capture using IIM, however, the DSP computer concatenates the samples. In this case you will need to use DSPWave.Next to access subsequent capture samples. So, in our example where both Cap1 and Cap9 are triggered before the samples are read back, the following can happen:

- Retrieving samples for Cap1 first will be successful, but to access samples for Cap9 you will need to issue a DSPWave.Next command. If you do not use a DSPWave.Next or if you try to access the capture samples using Cap9, you will get no samples back and will probably see a DSP error.
- Retrieving samples for Cap9 first will get samples for Cap1 first. To access samples for Cap9, you will need to issue a DSPWave.Next.

If you trigger capture waveform Cap1 and then retrieve its samples before triggering Cap9 no problems would occur in this example.

- DSP computer is disabled or no DSP computer is available on a tester with UltraVI80:

In this case, the UltraVI80 will not use IIM to process capture samples and will read from memory just as for the DC30/75. This means that multiple triggers on the same channel will only keep samples of the last triggered captures. (This is as opposed to using IIM, where multiple capture triggers would concatenate samples.) Prior triggered captures that have not been read back are overwritten. Using the example of Cap1 and Cap9, you may see the following problem:

- You trigger Cap1 then Cap9. If you then try to retrieve samples for Cap1, Cap9 samples will be returned instead. If you then try to retrieve samples for Cap9, you will see an error of Capture Not Triggered or No Samples Found.

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_program.3.11

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : Using DCDiffMeter in the Mixed Signal Workshop

Using DCDiffMeter in the Mixed Signal Workshop

The DCDiffMeter can be used in the Mixed Signal Workshop as a capture instrument. When a DCDiffMeter capture is added to the MSW, the following field appears in the Capture Instrument Block:

Type	This menu allows selection of the type of test this capture instrument block is programmed for. For UltraVI80 select DCDiffMeter5Timing. This determines which instrument capabilities are checked against.
------	---

For more information, see Mixed Signal Workshop.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_program.3.12

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : Visual Basic for Test

Visual Basic for Test

You can create DC instrumentation programs using VBT language in the VBT editor. Each instrument has its own properties and methods used for controlling the functionality of the instrument.

For more information, see [DCDiffMeter](#) in the VBT Syntax Reference.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_program.3.13

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Reset Values

UltraVI80 DCDiffMeter Reset Values

Resetting the connections disconnects the DCDiffMeter from the DUT. Resetting the settings clears the DCDiffMeter memory and returns the DCDiffMeter to the default settings.

Perform a reset using the following VBT syntax:

TheHdw.DCDiffMeter.Pins(PinList).Reset(tlResetConnections Or tlResetSettings)

The following is the reset state of DCDiffMeter parameters after a reset occurs:

- | | |
|---|------------------------------------|
| • | Stop the Capture |
| • | Set the Meter Voltage Range to 7v |
| • | Clear TCIO/Pattern collision error |
| • | Clear Missed Strobe error |
| • | Clear Missed Pset error |
| • | Clear the Memory Counter |
| • | Clear all Latched Alarms |
| • | Set Retrigger to Auto Rearm |

- [Set Slop to Positive Edge](#)
- [Disable the trigger to drive the AD](#)
- [Set the Trigger to Strobe](#)
- [Set the Trigger pin 1 to drive the AD](#)
- [Enable DGS Alarm](#)
- [Enable Over Range Alarm](#)
- [Reset the Meter Memory to 0](#)
- [Reset the Pset Memory Data to 0](#)
- [Reset the Pset Memory Addresses to 0](#)
- [Reset the Memory Write Pointer to 0](#)
- [Reset the Memory Read Pointer to 0](#)
- [Set DGS Select to 1, 3, 5, and 7 \(based on the group of 20 channels the DCDiffMeter is associated with\)](#)
- [Reset Memory Self Test to 0](#)
- [Reset the Sample Count to 0](#)
- [Reset the Sample Rate to 0](#)
- [Select Capture Memory to hold any measurement data \(Trigger or Strobe\)](#)

On the UltraVI80 when using VBT syntax, a DCDiffMeter is connected to an internal ground (calibration ground) whenever it is disconnected to prevent unwanted overvoltage alarms on reconnection. This connection to ground is also the default UltraVI80 DCDiffMeter connection state on power-up (to protect the DUT).

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_program.3.14

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Pattern Control

UltraVI80 DCDiffMeter Pattern Control

A pattern is used to precisely program an instrument at the vector level. You can program DCDiffMeter parameters using microcode or by applying PSets.

You create a pattern independently of the test program using the PatternTool or a text editor. Patterns are composed of a series of numbered digital vectors. These digital vectors define the data for each channel listed in the pattern, as well the pattern microcode used for controlling pattern execution.

The two forms of pattern control are:

Microcode	A microcode controls a single function, such as strobing a DCDiffMeter.
PSets	A PSet defines a programmed state for an instrument, so that a DCDiffMeter PSet contains parameters such as voltage range. When an instrument receives a PSet from a pattern, it enters the programmed state represented by that PSet.

The following topics are available.

- | | |
|---|--|
| • | Programming Patterns Using a Text Editor |
| • | Retrieving Strokes and Capture Data |
| • | Optimizing Pattern Restart Time |

	DCDiffMeter Pattern Microcodes
	PSets

For more information, see [DC Instrument Behavior During Pattern Execution](#).

[Pattern Compiler](#)

The [pattern compiler](#) takes a [text](#) .atp file and compiles it into a binary .pat file. This file can be edited using PatternTool and is the format accepted by the pattern loader.

[Pattern Reverse Compiler](#)

The [pattern reverse compiler](#) takes a [binary](#) .pat file and compiles it into a text .atp file. This file can be edited using a text editor.

[PatternTool](#)

[PatternTool](#) allows you to read and edit binary pattern files. Using [PatternTool](#), you can display a compiled pattern file in easily readable, column formats. You can also directly edit a pattern file, instead of having to edit the ASCII source file and recompile it. You can then update the tester with the edited pattern file.

[PatternTool](#) is useful during debugging, specifically in debugging pattern files.

<	>	
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_program.3.15

	
---	---

UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Pattern Control : Programming Patterns Using a Text Editor

Programming Patterns Using a Text Editor

You can create and edit a pattern using a text editor. This form (.atp file) must be compiled into a .pat file before it can be loaded.

Instruments Statement

To use a DCDiffMeter from within a pattern, you must add the pin to the instruments statement, which appears at the top of the file:

```
instruments = {
    MyPin:DCDiffMeter;
}
```

For more information, see [Instruments Statement](#) in the Pattern Language section.

Vector Statements

In a vector statement, program a DCDiffMeter pin like other instruments that use microcodes:

```
(MyPin = Strobe) > TSet ;
```

If a pin has multiple resources behind it, then the statement appears a bit differently, with the instrument type included:

```
(MyPin:DCDiffMeter = Strobe) > TSet ;
```

PatternTool is also used to edit patterns containing DC instruments.

Pattern Example

The following is an example of a pattern in .atp form created in a text editor for the DCDiffMeter.

```
import tset ts1;
instruments = {
    MyPin:DCDiffMeter 1;
}
```

```
srm_vector MyPattern ( $tset, MyDigitalPin)
{
(MyPin:DCDiffMeter = Filter_On)
    > ts1  0      ;
    > ts1  0      ;
(MyPin:DCDiffMeter = Strobe)
    > ts1  0      ;
repeat 1000    > ts1  0      ;
    > ts1  0      ;
(MyPin:DCDiffMeter = PSet MyPSet)
    > ts1  0      ;
    > ts1  0      ;
halt          > ts1  0      ;
}
```

The figure shows the .atp file compiled using the Pattern Compiler and loaded into PatternTool.

TSet	Command	Label	Vector/Cycle	MyPin (DCDiffMeter)	MyDigitalPin	Comment
ts1			0	Filter_On	0	
ts1			1		0	
ts1			2	Strobe	0	
ts1			3		0	
ts1	repeat 1000		4		0	
ts1			5	PSet MyPSet	0	
ts1			6		0	
ts1	halt		7		0	

Signal Names

You must define all signal names that appear in your pattern, and you must associate all signal names with a signal.

You do not need to define trigger names before you use them. You can retrieve the samples using VBT by referring to the name of the capture signal used in the pattern.

Public Function MyTest() As Long

' Get captured data

Dim data_captured As New PinListData

Dim data_wave As New DSPWave

data_captured = TheHdw.DCDiffMeter.Pins("MyPin").Capture _
 .Waveforms("MySignal").Item.DSPWave

' Site Loop to retrieve results for both sites

For Each Site In TheExec.Sites

data_wave = data_captured

data_wave.CalcMean ' or other DSP processing

Next Site

End Function

<		>	
	2015 Teradyne, Inc. All rights reserved.		

UltraVIDCDiffMeter_program.3.16

	
---	---

[UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Pattern Control](#) : Retrieving Strokes and Capture Data

Retrieving Strokes and Capture Data

When a meter is strobed, it stores a single measurement in meter memory. No name is associated with a stroke. Captures can consist of multiple measurements and are assigned a name. You define the characteristics of the capture, such as rate and number of samples, using a PSet preceding the Trig <name> microcode.

Use the following VBT syntax to retrieve a measurement strobed in a pattern:

Dim VMeasure As New PinListData

' Program the DCDiffMeter meter to make a stroke

VMeasure = TheHdw.DCDiffMeter.Pins("MyPin").Read(tlStroke, 1, 10000)

' Program limit options and datalog the results

' Compare results to limits in Flow Table

Call TheExec.Flow.TestLimit(resultVal:=VMeasure, ForceResults:=tlForceFlow)

Strokes and captures are handled differently when reading back. Strokes are available in a FIFO format directly from the DCDiffMeter drivers, using a ReadMeterNoStroke() function call. DSP must be used to retrieve capture data. For multiple captures, however, the captures are also retrieved in a FIFO format.

Note:

Although a test can be programmed to execute the stroke microcode at a 1s rate, exceeding the minimum capture rate of 120ps/sample in High Accuracy mode results in an ADC return value of zero.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_program.3.17

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Pattern Control : Optimizing Pattern Restart Time

Optimizing Pattern Restart Time

IG-XL can speed up restarts of patterns that contain DC instruments. If you have run a pattern and you do not use VBT to change any parameters that affect the pattern before the next start of the same pattern, enabling optimization allows the pattern to restart more quickly. Improvement can range from 20% to 50%. The first run of a pattern always requires the full amount of time to start. After you have run a pattern once, subsequent restarts (including starts from a different label in the same pattern) benefit from the reduced startup time.

To achieve this restart efficiency, the following parameters must remain constant from one start to the next:

- | | |
|---|-------------|
| • | Active site |
|---|-------------|
- | | |
|---|---|
| • | All DC PSet parameters (if you reprogram a PSet, do not apply it to hardware before you finish running the pattern) |
|---|---|
- | | |
|---|---|
| • | Source signal loaded in instrument memory |
|---|---|

Note that faster pattern restarts may require adjustments to test program timing.

To enable pattern restart optimization, set the `PatternRestartOptimizationEnabled` property to `True`.

Example

The following example runs the pattern three times. The first run, starting at label `t0`, requires the full startup time. The second run, a restart, and the third run, a start at a different label in the pattern, are both optimized.

***** Start at label t0 *****

```
If TheExec.Sites.ActiveCount > 0 Then
  TheHdw.Digital.Patterns.Pat("regulator").Start ("t0")
End If
TheHdw.Digital.Patgen.HaltWait
```

***** Do something that does not change any DC instrument settings *****

***** Restart at label t0 *****

```
If TheExec.Sites.ActiveCount > 0 Then
  TheHdw.Digital.Patgen.Restart
End If
TheHdw.Digital.Patgen.HaltWait
```

***** Start at label t1 *****

```
If TheExec.Sites.ActiveCount > 0 Then
  TheHdw.Digital.Patterns.Pat("regulator").Start ("t1")
End If
TheHdw.Wait (0.01)
TheHdw.Digital.Patgen.HaltWait
```

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_program.3.18

<

>

UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Pattern Control : DCDiffMeter Pattern Microcodes

DCDiffMeter Pattern Microcodes

DCDiffMeter pattern microcodes are instructions that you can place on an individual vector of a digital pattern. For that vector, a microcode instructs the DCDiffMeter to perform a particular action. A microcode controls a single function such as strobing the DCDiffMeter or enabling or disabling an alarm.

The format for specifying a DCDiffMeter microcode on a vector is either of the following:

- (pin-list:instrument = microcode)
- ((pin-list1:instrument, pin-list2:instrument) = microcode)

where:

pin-list	Is the pin name or pin group or (comma-separated list of pins) as specified in the instruments statement.
instrument	Is DCDiffMeter. This corresponds to the instrument type for pin-list as defined in the instruments statement.
microcode	A DCDiffMeter microcode.

The table lists the DCDiffMeter microcodes.

DCDiffMeter Microcodes	
Microcode	Description
Strobe	Strobes the meter and stores the measurement.

	When using the Strobe microcode, set the capture sample rate to a value other than 0 if you are using hardware averaging. See DCDiffMeter Hardware Averaging.
Enable_Inst_Cond	Enables the DCDiffMeter to drive the condition bit.
Disable_Inst_Cond	Disables the DCDiffMeter from driving the condition bit.
Enable_Alarm	Enables the alarm window.
Disable_Alarm	Disables the alarm window.
Clear_Cond	Clears the latched condition, while not disabling the condition bit from being driven by events on the DCDiffMeter.
Trig <name>	Starts a capture with the length and rate of the capture specified in the most recent PSet. See Trig Microcode.
Enable_ASynch_Trig	Enables an asynchronous trigger.
Disable_ASynch_Trig	Disables an asynchronous trigger.
PSet <name>	Applies a PSet of the specified name. See PSets.
Next_Capture_Sample	Captures the next sample to signal memory.

The UltraVI80 also supports microcode control of DCDiffMeter connections (through DCVI sense lines) using the following UltraVI80 DCVI microcodes:

- Connect_Diffmeter_High
- Disconnect_Diffmeter_High
- Connect_Diffmeter_Low
- Disconnect_Diffmeter_Low

For more information, see:

- Controlling a DCDiffMeter Connection from a Pattern
- DCVI Pattern Microcodes

For more information, see:

- Instrument Microcode

- Pattern Example

Controlling a DCDiffMeter Connection from a Pattern

The UltraVI80 supports microcode control of DCDiffMeter connections (through DCVI sense lines) using the following UltraVI80 DCVI microcodes:

- Connect_Diffmeter_High

- Disconnect_Diffmeter_High

- Connect_Diffmeter_Low

- Disconnect_Diffmeter_Low

However, controlling DCDiffMeter connections from a pattern is significantly different than using VBT syntax. When using these microcodes note the following:

- Microcode control of DCDiffMeter high and low side connections is autonomous. That is, the Connect_Diffmeter_High microcode only connects a high side pin, and the Connect_Diffmeter_Low only connects a low side pin. This is quite different than using VBT, where both high and low side DCDiffMeter connections are managed together.

For example, you would normally associate high and low side pins using a VBT statement such as:

```
TheHdw.DCDiffMeter.Pins("aHighSidepin").LowSide = "aLowSidepin"
```

This subsequently allows you to connect both pins to the DCDiffMeter associated with the high side pin by executing the following statement:

```
TheHdw.Pins("aHighSidepin").DCDiffMeter.Connect (tlDCDiffMeterConnectDefault)
```

You can make the same connection using microcode. However, you must use both a Connect_Diffmeter_High and a Connect_Diffmeter_Low microcode in the pattern using DCVI pins corresponding to the same sense lines as the pins used above.

- You must also define the corresponding DCDiffMeter high and low side pins in the channel map in a specific way when using these microcodes. Match the sense lines used in the channel map defining the DCDiffMeter high and low side pins with the sense lines that are defined for

the DCVI pins that are used in the pattern to apply the `Connect_DiffMeter_High/Low` and `Disconnect_DiffMeter_High/Low` microcodes.

- Microcode control of DCDiffMeter high and low side connections does not support connections across subbusses.

Referring to the VBT language example used above, if `aLowSidepin` was normally associated with a different DCDiffMeter than `aHighSidepin`, a cross subbus connection would be made to route `aLowSidepin` to the same DCDiffMeter as is associated with `aHighSidepin`. Under microcode control, however, the high or low side connection is made directly to the DCDiffMeter associated with the pin.

- Since microcode control of DCDiffMeter high and low side connections is autonomous, an initial high and low side connection can be established using VBT language before pattern execution and only the high (or low) side connection can be changed during pattern execution. This is particularly important to remember since:

- Low side DGS pins cannot be specified in a pattern

- Connections across subbusses cannot be established during pattern execution (although existing subbus connections made using VBT language are maintained)

- You can define two DCDiffMeter pins in a channel map that both point to the same DCDiffMeter. However, you must use the same pin to connect and disconnect from that DCDiffMeter. For example, you might define pins `DVMSense1` and `DVMSense2`, both of which point to `DCDiffMeter1`. In this case, if you use `DVMSense1` to connect, you must use the same pin, `DVMSense1`, to disconnect `DCDiffMeter1`. You cannot use `DVMSense2` to disconnect this DCDiffMeter.

- DCDiffMeter VBT language checks for connection conflicts before making a connection. This does not happen under microcode control. Previous connections are broken without warning.

- On the UltraVI80 when using VBT language, a DCDiffMeter is connected to an internal ground (calibration ground) whenever it is disconnected to prevent unwanted overvoltage alarms on reconnection. This is not the case, however, under microcode control. Thus, overvoltage

alarms may be experienced under certain conditions requiring a disconnect microcode to be immediately followed by a connect.

- On pattern completion, all disconnected DCDiffMeters will be connected to an internal ground to prevent unwanted overvoltage alarms. This is also the default UltraVI80 DCDiffMeter connection state on power-up (to protect the DUT).

For more information, see [DCVI Pattern Microcodes](#).

DCDiffMeter PatternTool GUI

The PatternTool provides columns for the DCDiffMeter pins listed in the instrument statement of the compiled .atp file.

You can conveniently produce the comma-separated microcode equivalent by checking the check boxes as in shown in the figure.

DCDiffMeter Microcode Display

For more information, see:

- [Pattern Example](#)

- [About PatternTool](#)

Trig Microcode

The DCDiffMeter accepts the Trig <name> syntax pattern. You assign names to signals in Mixed Signal workshop or using VBT. IG-XL ensures that all signals used by a pattern in SRM are loaded when the pattern is loaded into pattern memory. Each channel has memory for eight signal names. If you define and load more than eight signals per channel per pattern, IG-XL returns an error when loading the pattern.

For more information, see [Mixed Signal Workshop](#).

<		>	
	2015 Teradyne, Inc. All rights reserved.		

UltraVIDCDiffMeter_program.3.19

<

>

UltraVI80 DCDiffMeter Programming : UltraVI80 DCDiffMeter Pattern Control : PSets

PSets

A PSet allows you to specify several aspects of an instrument’s state at once, such as the force voltage value and force voltage range. PSets are programmable in the VBT code and applied using either VBT language or the PSet microcode. The DCDiffMeter supports PSet memory for a total 62 PSets per pattern. This includes any DCDiffMeter, DCVI, and DCTime PSets that may be stored in memory as well.

PSet properties for the DCDiffMeter are listed in the table below.

DCDiffMeter PSets	
DCDiffMeter PSet Property	Description
Capture.SampleRate	Set the DCDiffMeter capture sample rate.
Capture.SampleSize	Set the DCDiffMeter capture sample size.
Capture.ExtraSamples	Set the DCDiffMeter capture extra samples.
ConditionBit	Controls the event that drives the condition bit. This property is only important for patterns.
VoltageRange	Programs the voltage range.
HardwareAverage	Sets the number of samples to be averaged by this DCDiffMeter during measurements. This number must be a power of 2. Maximum is 256.

For more information, see [TheHdw.DCDiffMeter.Pins.PSets](#).

[PSet Programming Example](#)

You program a PSet using PatternTool or a text editor. You define a PSet in VBT, as in the following example:

' Add the PSet to the PSet collection

```
TheHdw.DCDiffMeter.Pins("MyPin").PSets.Add "PSet1"
```

' Program DCDiffMeter instrument parameters for the PSet

```
TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("PSet1").ConditionBit = False
```

```
TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("PSet1").VoltageRange = 7
```

' Apply the programmed settings to the hardware

```
TheHdw.DCDiffMeter.Pins("MyPin").PSets("PSet1").Apply
```

You can also add a PSet to the collection as follows:

```
TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("MyPSet").Set 7, False, 1000 _  
    , 300, 3, 64
```

Applying a PSet

You apply a PSet from either a pattern or to the instrument in VBT. To apply a PSet with VBT, use the following syntax:

```
TheHdw.DCDiffMeter.Pins("MyPin").PSets.Item("MyPSet1").Apply
```

A PSet called from a text file would use the following format:

(MyPin:DCDiffMeter = PSet PSet1)

To call a PSet in PatternTool the setup would look as follows:

TSet	Command	Label	Vector/Cycle	Comment	ch1hi (DCDiffMeter)
-			0		
-			1		PSet PSet1
-			2		
-	repeat 500		3		
-			4		

PSet Called Using PatternTool

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_too.2.01

	
---	---

UltraVI80 DCDiffMeter Theory of Operation

UltraVI80 DCDiffMeter Theory of Operation

An UltraVI80 DC Differential Voltmeter (DCDiffMeter) is a differential meter that performs small voltage measurements on large common-mode signals. A DCDiffMeter has three lower voltage ranges with improved accuracy specifications over the UltraVI80 board DCVI meters. An UltraVI80 board has eight DCDiffMeters.

A DCDiffMeter can be asynchronously triggered to perform a measurement by listening to one of the four onboard DC Time Measurement Unit (DCTime) trigger lines.

The high and low sides connect to the DUT using the channel sense lines. The sense lines that can be used depend on the instrument board and DCDiffMeter being connected.

The following topics are available:

- [DCDiffMeter Overview](#)
- [DCDiffMeter Subbus Connections](#)
- [DCDiffMeter Metering and Capture](#)
- [DCDiffMeter Alarms](#)
- [DCDiffMeter Errors](#)

For more information, see [Connecting a DCDiffMeter](#).

For information on DCDiffMeter instrument performance specifications and standards, see [UltraVI80 Specifications](#).

< >		
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_too.2.02

<

>

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Overview

DCDiffMeter Overview

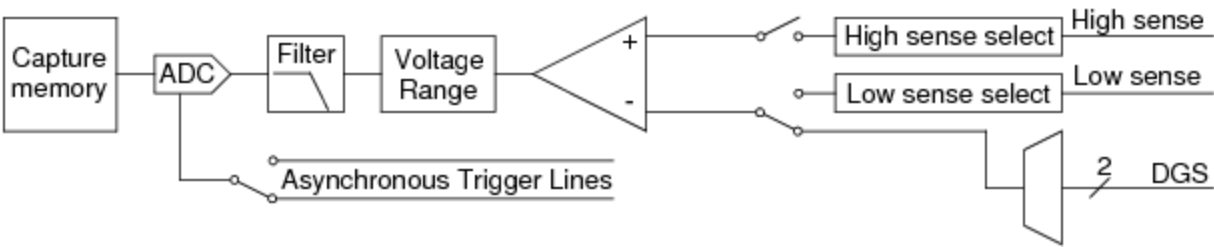
A DC Differential Voltmeter performs differential voltage measurements by connecting the high side to a DUT pin and the low side to a separate DUT pin or DGS.

The DCDiffMeter connects using a channel sense line for both the high and low side. The channel sense line that is used depends on what DCDiffMeter is being used and on other DUT connections.

The low side can also connect to either of two DGS lines.

The DCDiffMeter includes a filter that allows you to attenuate high frequency noise that may cause deviations in measurements.

The figure shows a simplified block diagram for a DCDiffMeter.



DCDiffMeter Block Diagram

For more information, see [DCDiffMeter Subbus Connections](#).

<

>

UltraVIDCDiffMeter_too.2.03

<

>

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Overview](#) : DCDiffMeter Modes

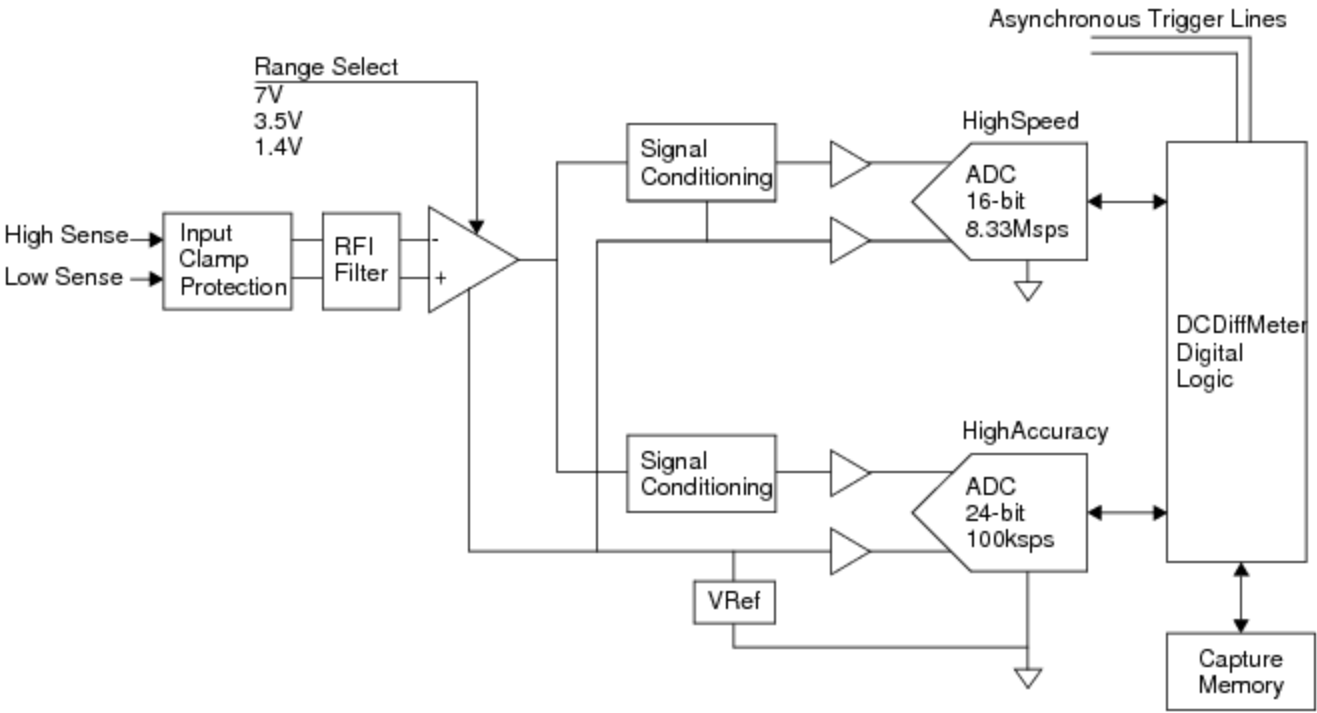
DCDiffMeter Modes

UltraVI80 Differential Voltmeter channels are similar to those for the DC30. To improve accuracy, however, the 16-bit SAR ADC used on the DC30 is replaced with two separate analog-to-digital converters, available in two modes of operation, High Accuracy and High Speed.

Comparison of DCDiffMeter Modes

Feature	High Speed	High Accuracy
Accuracy	16-bit ADC	24-bit oversampled SAR ADC
Strobe Memory	1k	1k
Waveform Samples Memory	4M	Waveform captures not supported
Sample Rate	Up to 8.33Msps adjustable, 120ns resolution	500sps to 100ksps adjustable, 10s resolution
Voltage Ranges	1.4V, 3.5V, and 7V	1.4V, 3.5V, and 7V
Hardware Averaging	Yes	Yes

The following is a block diagram of the two types of DCDiffMeter channels.



DCDiffMeter Channel Block Diagram

<		>
2015 Teradyne, Inc. All rights reserved.		

UltraVIDCDiffMeter_too.2.04



UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Subbus Connections

DCDiffMeter Subbus Connections

An UltraVI80 instrument board has eight DC Differential Voltmeters. Each DCDiffMeter connects to a set of sense channel lines.

The eight DCDiffMeters on each board reside on four differential subbuses. This means that each subbus has two DCDiffMeters that have access to 20 channels. Each UltraVI80 DCDiffMeter has direct access to 10 channels and indirect access to the other 10 channels.

Each channel sense pin connects directly to half the channels (10 channels) on one of the four differential subbuses. Measurements can be made between any two DUT pins on the 20 channels connected to that subbus. Each DCDiffMeter can make a simultaneous measurement on any two channels located on the same subbus, and in the same 10 channel group, as the measuring DCDiffMeter.

If a DCDiffMeter is indirectly connected to a channel on its differential subbus, only one DCDiffMeter measurement can be made at a time. The figure shows that this is due to the subbuses connecting and the need for the DCDiffMeters to share subbuses.

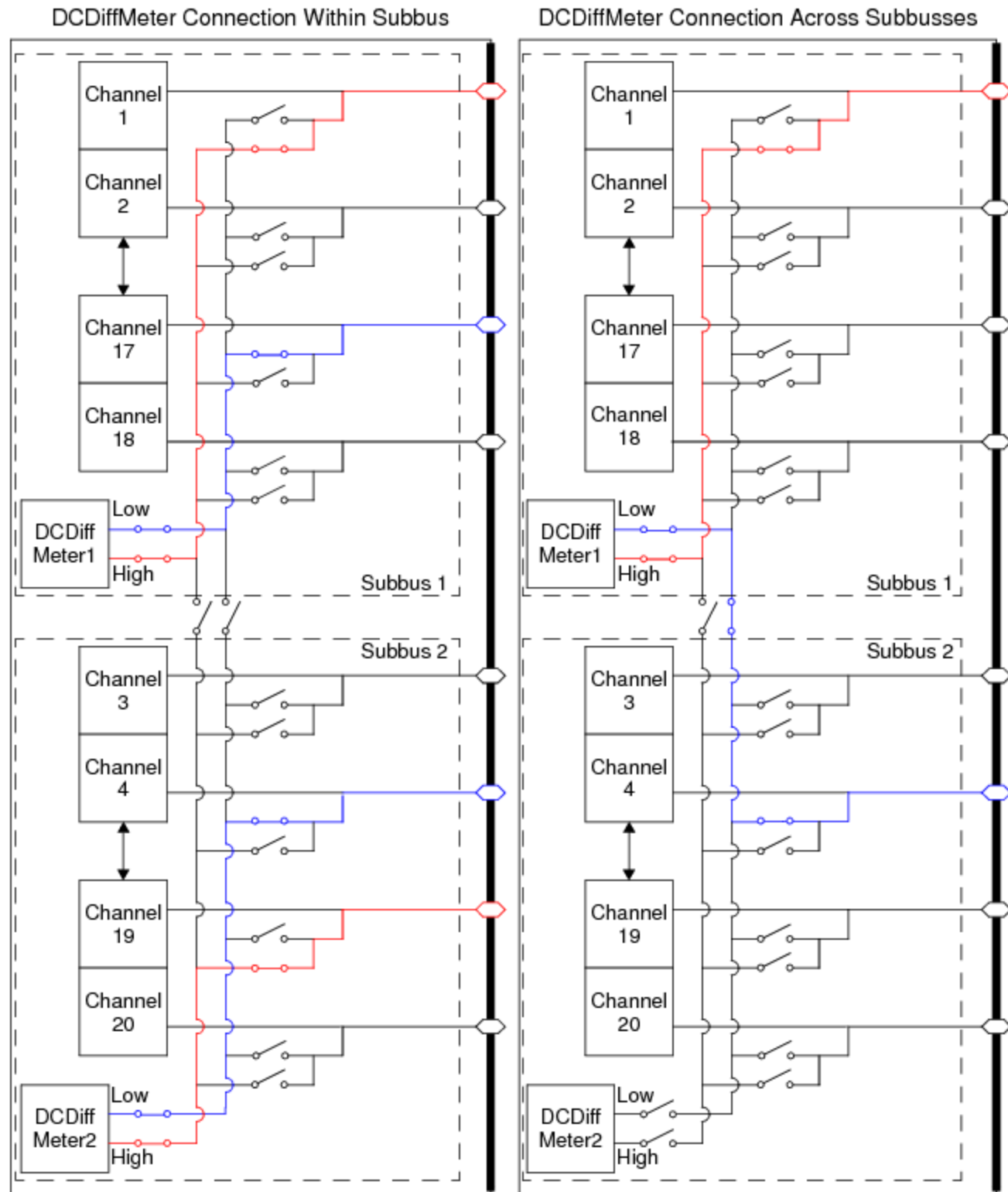
The DCDiffMeter that connects to the channels is based on the high side connection. If the high side is connected to a channel that DCDiffMeter 1 has a direct connection to, then DCDiffMeter 1 is used. If the high side connection is made to a channel that DCDiffMeter 2 has a direct connection to, then DCDiffMeter 2 is used.

Note:

Choose the DIB layout carefully to take advantage of parallel differential measurements.

In the figure, the diagram on the left displays DCDiffMeter 1's high side connected to channel 1, and the low side is connected to channel 17 through subbus 1. On subbus 2, a second simultaneous differential meter measurement is being made. The DCDiffMeter 2's high side is connected to channel 19, and the low side is connected to channel 4.

The diagram on the right shows DCDiffMeter 1's high side connected to channel 1, while the low side is connected to channel 4 through the subbus. Since the DCDiffMeter is connecting through the subbus, only one DCDiffMeter measurement can be made at a time.



UltraVI80 DCDiffMeter Connections, Channels 1 - 20

For more information, see:

- [Connecting a DCDiffMeter](#)

- [UltraVI80 Channel Connections](#)
-
- [DCDiffMeter DGS](#)

Potential Conflicts with Subbus Connections

Certain combinations of DC Differential Meter connections have the potential to create subbus conflicts, which can affect measurements. For example, consider the following example:

- [Connect DCDiffMeter1 across the subbus \(Hi to channel 1 and Lo to channel 4\)](#)
-
- [Connect DCDiffMeter2 across subbus \(Hi to channel 3 and Lo to channel 4\)](#)

This combination has the potential to short channel 2 to channel 4 through the sbusLo1 to sbusLo2 connection, which may provide inaccurate measurements.

IG-XL 8.00.00 and later includes a check to determine if the subbus associated with a DCDiffMeter and with the low channel is already in use. If the low side bus is already in use, IG-XL returns the following error:

Cannot program the LowSide pin if the DCDiffMeter is already connected. Disconnect the currently connected DiffMeter pin(s) before changing the LowSide pin parameter.

To avoid getting this error, use the property `.LowSideCrossSBusConnectEnable` to disable low side subbus error messages.

For more information, see [LowSideCrossSBusConnectEnable](#) in the VBT Syntax Reference.

UltraVIDCDiffMeter_too.2.05

<	>
-----------------	-----------------

[UltraVI80 DCDiffMeter Theory of Operation](#) : DCDiffMeter Metering and Capture

DCDiffMeter Metering and Capture
The following topics are available:

- | | |
|---|--|
| • | DCDiffMeter Metering Ranges |
| • | Waveform Capture |
| • | DCDiffMeter Hardware Averaging |
| • | Asynchronous Trigger |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.06

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Metering and Capture](#) : DCDiffMeter Metering Ranges

DCDiffMeter Metering Ranges

Each DCDiffMeter has ranges for voltage measurements that are provided to increase measurement accuracy. UltraVI80 DCDiffmeter ranges are: 1.4V, 3.5V, and 7V. You can set ranges manually using VBT or PSets, or IG-XL can choose the best range using autoranging. Note that the accuracy specification for each DCDiffMeter range in High Speed mode (16 bit ADC) is split into two segments. Accuracy is specified for the following values in each voltage range:

•	1.4V range
–	1.0V
–	1.0 - 1.4V
•	3.5V range
–	2.5V
–	2.5 - 3.5V
•	7.0V range

–	5.0V
---	------

–	5.0 - 7.0V
---	------------

For more information, see [DC Differential Voltmeter](#) in UltraVI80 Hardware Specifications.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.07

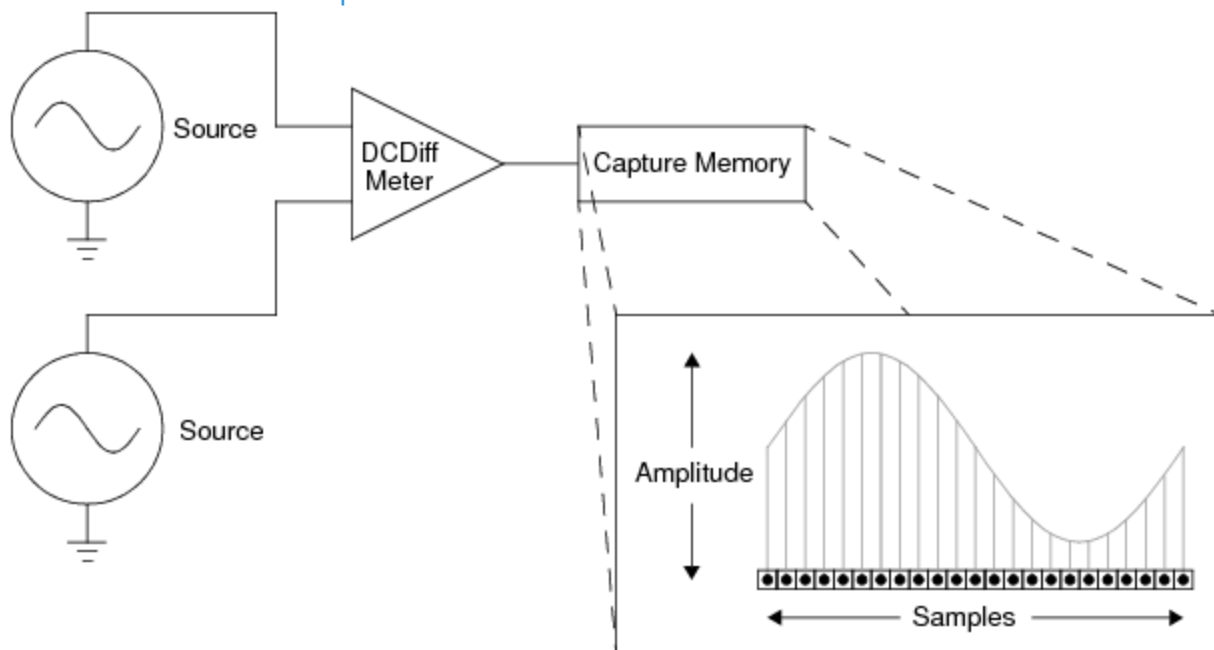


UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Metering and Capture : Waveform Capture

Waveform Capture

Each DCDiffMeter can capture up to 4M samples of a waveform (in High Speed mode) with a capture rate of 508.66Hz to 8.34MHz.

This figure shows a waveform being captured. The signal is converted to a series of points along the waveform and stored in capture memory. The points captured are based on the capture rate and the number of samples.



Waveform Capture Using DCDiffMeter

Each time the meter captures a value, it is stored in memory. A counter tracks how many values have been stored. If more than 4M samples are stored, the memory overwrites previously stored data and a data lost alarm is generated.

Capture Controls

You can trigger or stop a capture using VBT language or the DCDiffMeter debug display.

< >		
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_too.2.08

<	>
---	---

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Metering and Capture](#) : DCDiffMeter Hardware Averaging

DCDiffMeter Hardware Averaging

The UltraVI80 DCDiffMeter can perform hardware averaging for measurements. Using hardware averaging, a DCDiffMeter meter can:

- Measure a number of samples (2, 4, 8, 16, 32, 64, 128, or 256 samples) at a fixed sampling rate of 8.33Msps (0.12s between samples) in High Speed mode or 100ksps (10s between samples) in High Accuracy mode.
- Calculate the average of the samples
- Store the average in Strobed Memory

This method allows you to acquire samples and calculate an average in hardware faster than using VBT syntax to perform software averaging.

For example, you could use the following statement to make 256 measurements at a sampling rate of 8.33Msps in High Speed mode on pin MyPin, have these measurements read back and averaged by software, and finally return the averaged value.

Dim Results As New PinListData

TheHdw.DCDiffMeter.Pins("MyPin").HardwareAverage=1

Results = TheHdw.DCDiffMeter.Pins("MyPin").Read (tlStrobe, 256, 8330000, _
tlDCDiffMeterReadingFormatAverage)

You could do the same thing using hardware instead of software averaging as follows:

Dim Results As New PinListData

TheHdw.DCDiffMeter.Pins("MyPin").HardwareAverage=256

```
Results = TheHdw.DCDiffMeter.Pins("MyPin").Read (tlStrobe, 1, 8330000, _
tIDCDiffMeterReadingFormatAverage)
```

In this example, 256 samples are read back from memory when using software averaging, and only one sample is read back when using hardware averaging.

The default for the `.HardwareAverage` property is 1. You can also use hardware averaging in a PSet. Note that the sampling rate for hardware averaging is fixed at 8.33Msps in High Speed mode and 100ksps in High Accuracy mode. The rate is not affected by sampling rates set for capture methods using strobes and signals. So, for example, you can use the following VBT statements to set the capture sampling rate:

```
TheHdw.DCDiffMeter.Pins.Read
```

```
TheHdw.DCDiffMeter.Pins.Capture.SampleRate
```

```
TheHdw.DCDiffMeter.Pins.Strobe
```

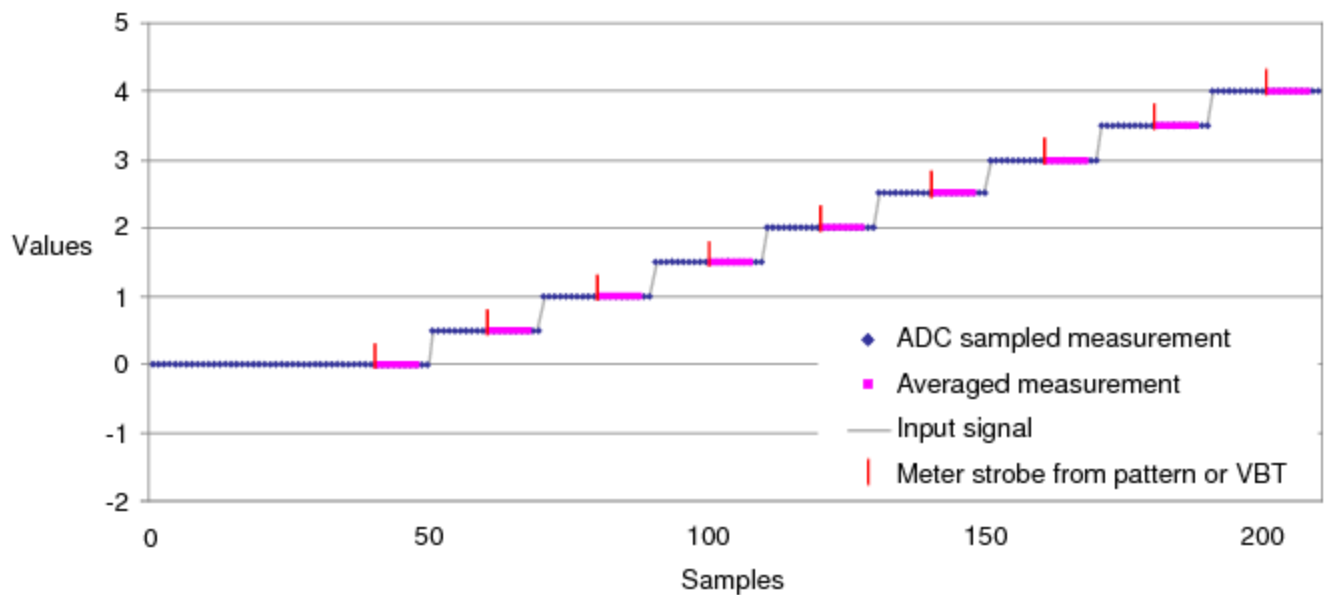
However, these statements do not affect the fixed rate of sampling for hardware averaging.

For more information, see:

- [TheHdw.DCDiffMeter.Pins.HardwareAverage](#)
- [TheHdw.DCDiffMeter.Pins.PSets.Item.HardwareAverage](#)

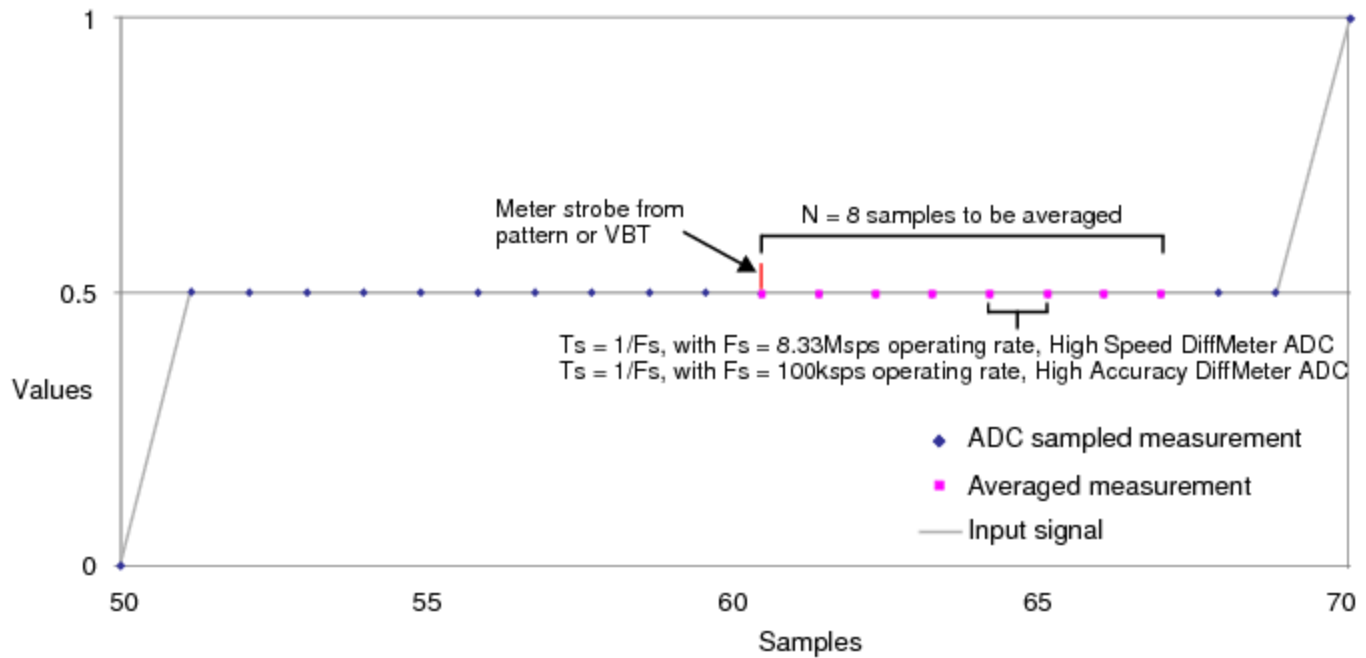
Hardware Averaging Using Strobes

The following figure shows an example of a signal (DC Levels) acquired using Meter strobes with hardware averaging set to 8.



DCDiffMeter Strobes Hardware Averaging

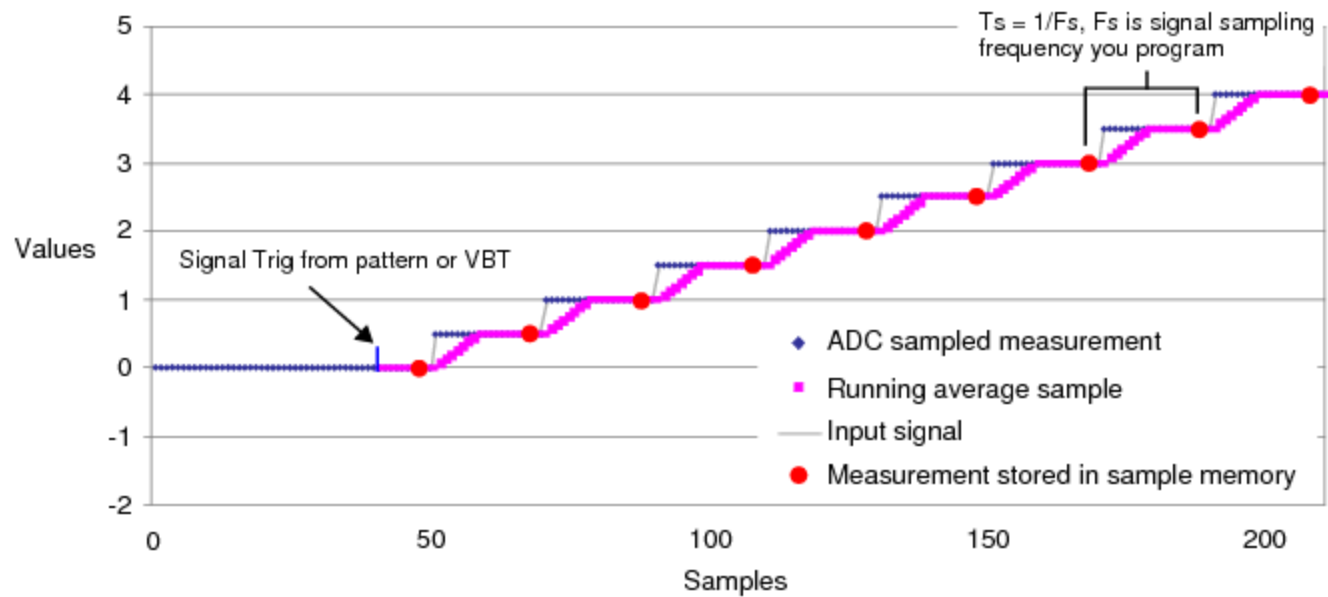
The following figure is the same example, but in more detail showing 20 samples.



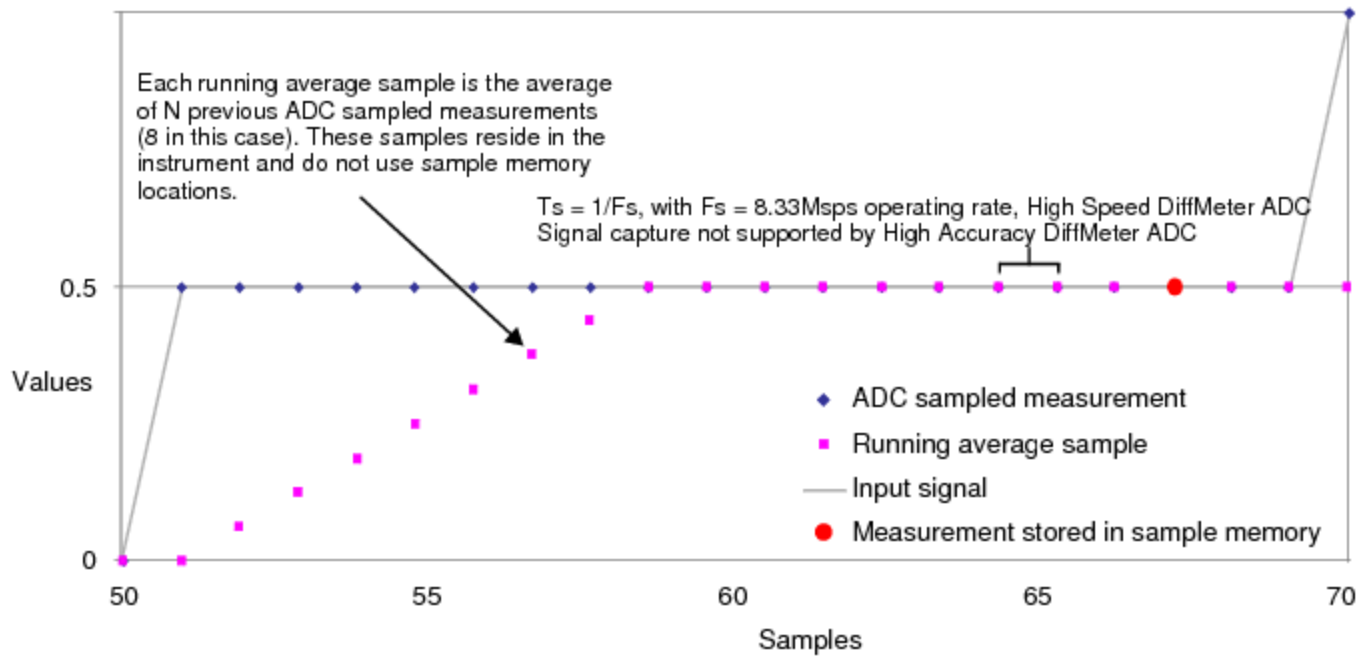
DCDiffMeter Strokes Hardware Averaging, Detail Showing 20 Samples

Hardware Averaging Using Signals

The following figures show an example of a signal (DC Levels) acquired using a Signal with hardware averaging set to 8. Note that signal capture is not supported in High Accuracy mode for DCDiffMeter.



DCDiffMeter Signal Hardware Averaging



DCDiffMeter Signal Hardware Averaging, Detail Showing 20 Samples

<

>

2015 Teradyne, Inc. All rights reserved.

UltraVIDCDiffMeter_too.2.09

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Metering and Capture](#) : Asynchronous Trigger

Asynchronous Trigger

Measurements and captures can be initiated asynchronously by way of the DUT using the asynchronous trigger feature of the DCDiffmeter. An UltraVI80 DCDiffMeter can be triggered by a DCTime only if they both reside in the same group of 40 channels (1-40 or 41-80).

For more information, see:

- [Asynchronous Trigger](#)
- [Onboard Asynchronous Triggering of a DCDiffMeter](#)

<	>	
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_too.2.10

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Alarms

DCDiffMeter Alarms

An alarm indicates that hardware is not operating as expected given the way an instrument is programmed. It indicates that a condition exists that could potentially invalidate a measurement. The DCDiffMeter alarms are:

- | | |
|---|---------------------|
| • | Capture |
| • | CaptureNotTriggered |
| • | DataLost |
| • | DGS |
| • | MeterMemoryFull |
| • | Overrange |
| • | OverVoltage |
| • | SignalMemoryFull |
| • | Timeout |

< >		
	2015 Teradyne, Inc. All rights reserved.	

UltraVIDCDiffMeter_too.2.11

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Alarms : Capture

Capture

An alarm occurred during a capture and the sample latched an alarm. Any of the DCDiffMeter alarms could have occurred while the capture was running.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0006 : Instrument alarmed during capture.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- An alarm condition occurred during the capture and may cause a measurement to be invalid.
 - Based on the alarm, the instrument may be in a different condition than expected.

<	>
-----------------	-----------------

UltraVIDCDiffMeter_too.2.12

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Alarms](#) : CaptureNotTriggered

CaptureNotTriggered
Requested data on a sample was never triggered.
Alarm Message
The following is the datalog alarm message:
{-1} : alarm [PinName, SiteNum] DDM : 0009 : Capture not triggered.
where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.13

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Alarms](#) : DataLost

DataLost

The capture overwrote the previous existing data due to a readback not occurring and no room was left in memory to retain all data.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0007 : Data lost. Memory was overwritten by subsequent captures.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.14

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Alarms : DGS

DGS

Identifies a difference in voltage between the DGS wire and DUT ground.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0004 : DGS alarm in the hardware.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | |
|----------------------------|
| <div>• Miswired DIB.</div> |
|----------------------------|

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.15

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Alarms](#) : MeterMemoryFull

MeterMemoryFull

The meter memory has overflowed.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0060 : Data lost. Meter Memory Full Alarm occurred due to subsequent strobes or meter read.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- Not enough meter reads being done to empty meter memory for subsequent meter strobes.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.16

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Alarms : Overrange

Overrange

The input to the A/D converter in the voltmeter or ammeter is greater than the full-scale programmed range. The selected voltage or current range is too low.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0005 : Overrange alarm in the hardware.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | | |
|---|----------------------------|
| • | Incorrect range programmed |
| • | Defective device |

<	>
-----------------	-----------------

UltraVIDCDiffMeter_too.2.17

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Alarms : OverVoltage

OverVoltage

A DCDiffMeter pin Voltage is outside of the common mode range. Protected to 30V.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0059 : Overvoltage alarm. Voltage is outside of common mode range.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- | | |
|--|--|
| | <ul style="list-style-type: none">Incorrect voltage programmed |
| | <ul style="list-style-type: none">Defective device |

<	>
-----------------	-----------------

UltraVIDCDiffMeter_too.2.18

<	>
-----------------	-----------------

UltraVI80 DCDiffMeter Theory of Operation : [DCDiffMeter Alarms](#) : SignalMemoryFull

SignalMemoryFull

The signal memory has overflowed.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0061: Data lost. Signal Capture Memory Full Alarm occurred due to subsequent signal captures.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- Captured signals are not being readback/processed quickly enough to allow other captures to occur.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.19

<

>

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Alarms : Timeout

Timeout

The capture data was not available when the instrument was trying to retrieve it.

Alarm Message

The following is the datalog alarm message:

{-1} : alarm [PinName, SiteNum] DDM : 0008 : Timeout occurred while retrieving data.

where:

PinName	Name of the pin the alarm occurred on.
SiteNum	Site number the alarm occurred on.

Common Causes

- The data being retrieved took longer than the timeout setting.
- There is no data to retrieve.
- Two samples were requested but only one was taken. The hardware waits for the second sample for up to 100ms or the programmed timeout.

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

UltraVIDCDiffMeter_too.2.20

<

>

UltraVI80 DCDiffMeter Theory of Operation : DCDiffMeter Errors

DCDiffMeter Errors

An error is a condition that is determined by the hardware at run time and does not represent a risk of hardware damage. An error occurs when the hardware does not enter a programmed state or does not initiate a requested process.

The DCDiffMeter can return the following errors:

Cap Param Changed [Capture Parameter Changed]	Some parameters were changed while a capture was in progress. The change is not permitted.
Missed PSet	Missed PSet due to a timing issue. Examine your pattern and adjust the timing as needed.
Missed [Meter] Strobe	Missed Strobe due to a timing issue. Examine your pattern and adjust the timing as needed.
TCIO Patt Coll [TCIO and Pattern Collision]	Pattern and TCIO collision detected. This indicates a collision between commands coming from the host computer and the pattern generator.
Too Many Next Cap [uCodes]	Too many Next Capture microcodes executed on the segment size. Examine your patterns and adjust them as needed.
Trigger with High Accuracy	A Trigger command was executed with High Accuracy Mode selected. Select High Speed Mode when using the Trigger microcode or use the Strobe microcode.

<		>	
	2015 Teradyne, Inc. All rights reserved.		